

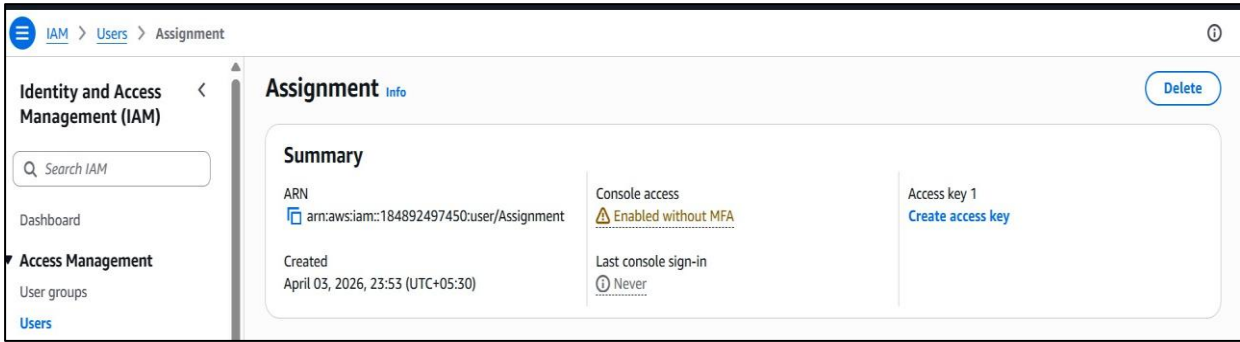
Graded Assignment

Cloud Services & Platforms

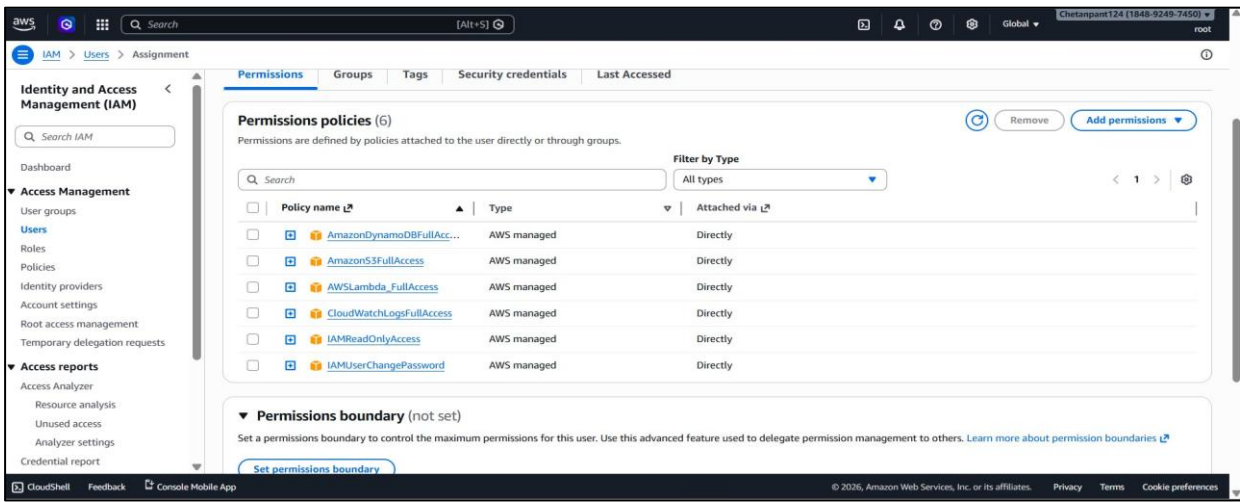
Part A : IAM Setup

A dedicated IAM user named **Assignment** was created from the root account with console access enabled. All remaining work was completed exclusively using this non-root user, in line with AWS security best practice. Permissions were scoped strictly to what the task required - following the **principle of least privilege**. Five policies were attached directly: AmazonS3FullAccess, AWSLambda_FullAccess, AmazonDynamoDBFullAccess, CloudWatchLogsFullAccess, and IAMReadOnlyAccess. No broad access such as AdministratorAccess was granted.

Fig 1 — IAM User Summary & Permissions



Assignment user summary - ARN, console access enabled, creation date



Least-privilege policies attached to Assignment user

Part B : Architecture Design

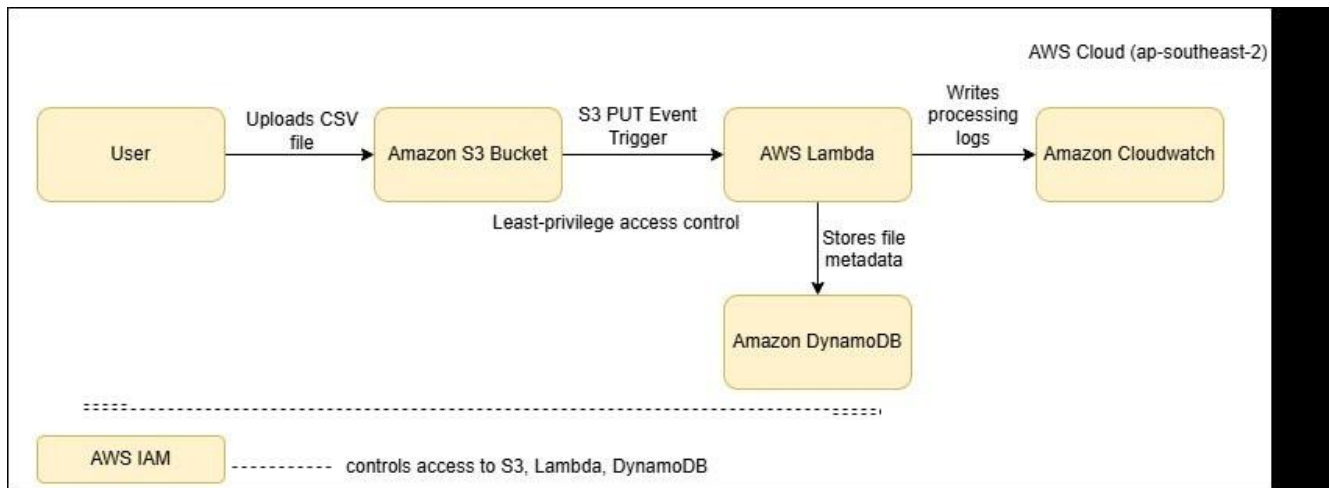


Fig 2: Serverless data processing pipeline — AWS Cloud (ap-southeast-2)

How the Services Interact

The pipeline begins when a user uploads a CSV file to the Amazon S3 bucket. S3 detects the upload automatically and invokes the Lambda function via a PUT event notification no polling, no manual trigger. Lambda reads the file directly from S3, extracts metrics (file name, type, size, row count, column count), logs everything to CloudWatch, and writes a structured record into DynamoDB. Each service hands off to the next without any human intervention after the initial upload.

How Security Is Enforced

Security operates at two levels. The Assignment IAM user holds only the five permissions this task requires root was used solely for initial setup. Lambda runs under its own execution role, **LambdaFileProcessingRole**, which grants S3 read, DynamoDB write, and CloudWatch log access and nothing else. Separating user permissions from service permissions is a core principle of secure cloud design. The S3 bucket was also kept private with all public access blocked.

Why Event-Driven Design Was Used

A traditional approach needs a server running continuously, polling S3 for new files even when none arrive wasting compute and money. Event-driven design solves this cleanly. Lambda runs only when S3 fires a trigger, consuming resources only when real work exists. It also scales automatically whether one file or a hundred arrive simultaneously, each is handled independently with no manual configuration required.

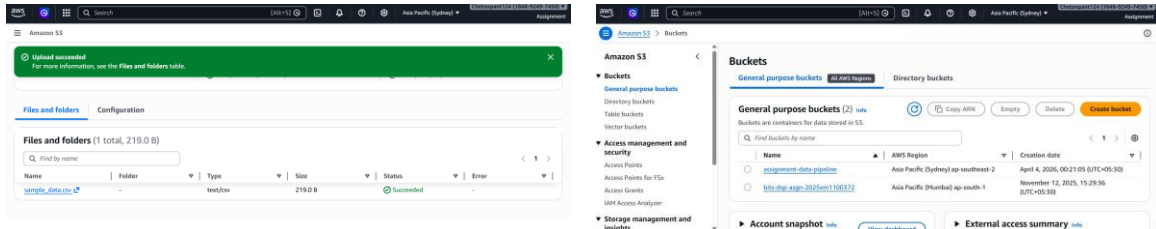
Part C : AWS Resource Setup

C1 : Amazon S3 Bucket

Bucket **assignment-data-pipeline** was created in ap-southeast-2 (Sydney) with all public access blocked. A CSV dataset (sample_data.csv 5 rows, 5 columns, 219 bytes) was uploaded to trigger the pipeline.

C2 : Amazon DynamoDB Table

Table **FileProcessingMetadata** was created with partition key **file_name (String)**. The table started empty and was populated automatically by Lambda on each upload no manual data entry at any point.



Left: S3 upload succeeded — sample_data.csv (219 B) | Right: DynamoDB table FileProcessingMetadata — Active, partition key file_name (S)

C3 : AWS Lambda Function + S3 Trigger

Function **FileProcessingFunction** (Python 3.12) was created with the provided baseline script deployed without modification. The function was assigned **LambdaFileProcessingRole** (pre-created as root) and configured with S3 as the event trigger on PUT events.

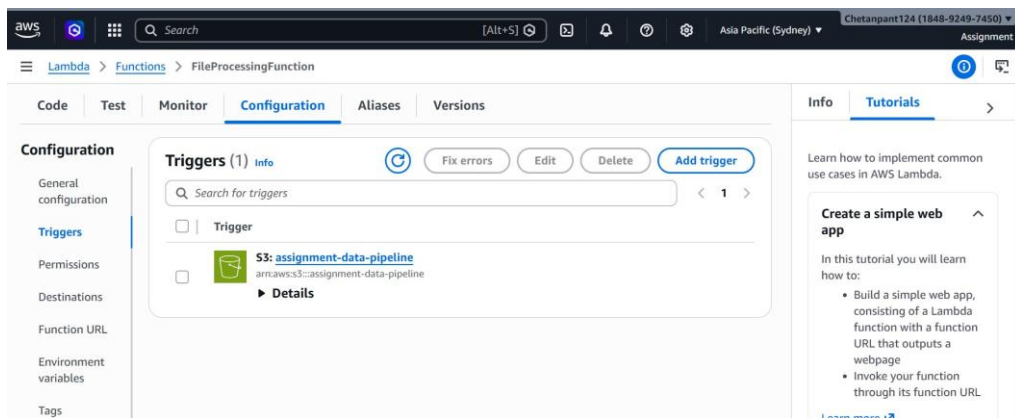
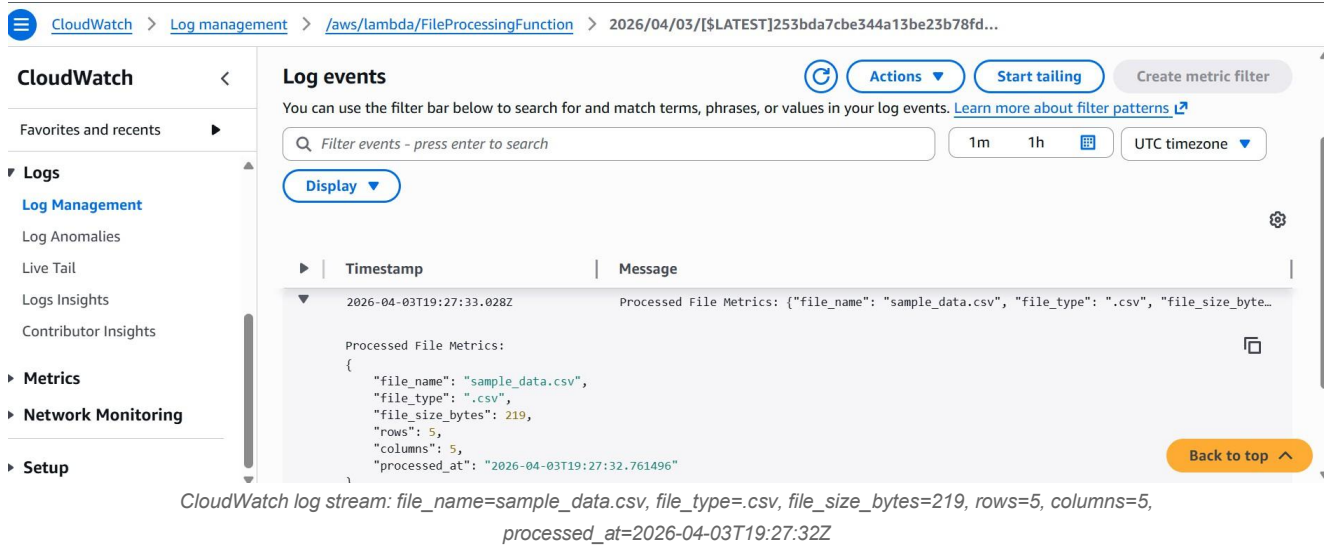


Fig 5: S3 bucket 'assignment-data-pipeline' configured as Lambda trigger

Part D : Output Validation

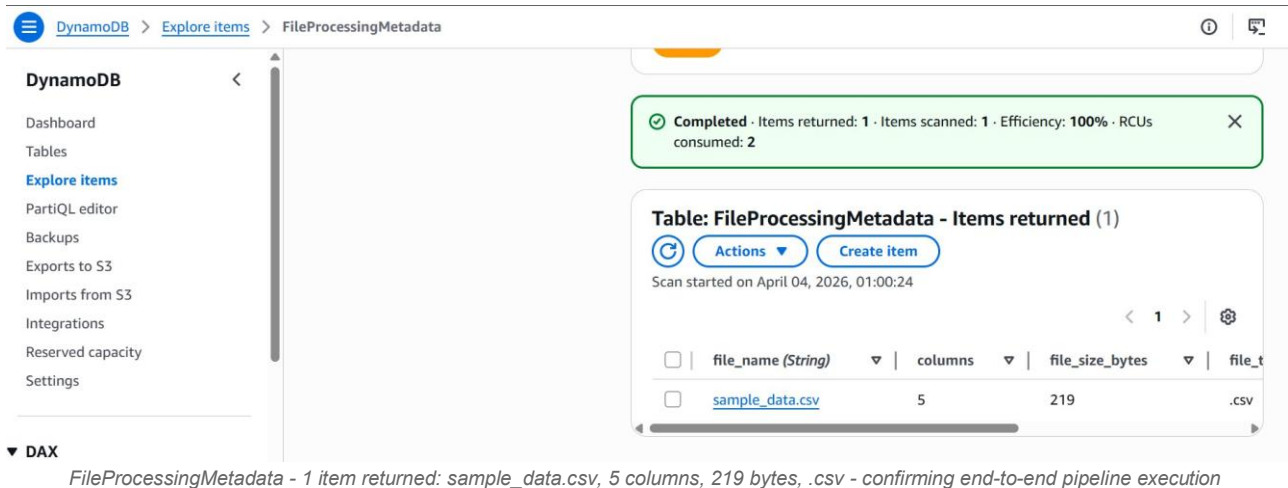
After uploading `sample_data.csv` to S3, Lambda was invoked automatically within seconds via the S3 PUT event. The function extracted file metrics, logged them to CloudWatch, and wrote a record into DynamoDB entirely without manual intervention. Both outputs are shown below.

Fig 6 : CloudWatch Log Output



The screenshot shows the AWS CloudWatch console interface. The breadcrumb navigation at the top reads: CloudWatch > Log management > /aws/lambda/FileProcessingFunction > 2026/04/03/[\$LATEST]253bda7cbe344a13be23b78fd... The left sidebar contains navigation links for CloudWatch, Favorites and recents, Logs, Log Management, Log Anomalies, Live Tail, Logs Insights, Contributor Insights, Metrics, Network Monitoring, and Setup. The main panel is titled 'Log events' and includes a search bar, a 'Display' button, and filters for time range (1m, 1h) and timezone (UTC). A log event is expanded, showing a timestamp of 2026-04-03T19:27:33.028Z and a message containing JSON data: Processed File Metrics: { "file_name": "sample_data.csv", "file_type": ".csv", "file_size_bytes": 219, "rows": 5, "columns": 5, "processed_at": "2026-04-03T19:27:32.761496". Below the screenshot, the text 'CloudWatch log stream: file_name=sample_data.csv, file_type=.csv, file_size_bytes=219, rows=5, columns=5, processed_at=2026-04-03T19:27:32Z' is displayed.

Fig 7 : DynamoDB Table Entry After Upload



The screenshot shows the AWS DynamoDB console interface. The breadcrumb navigation at the top reads: DynamoDB > Explore items > FileProcessingMetadata. The left sidebar contains navigation links for DynamoDB, Dashboard, Tables, Explore items, PartiQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, and Settings. The main panel shows a 'Completed' status bar indicating 1 item returned and 100% efficiency. Below this, a table titled 'Table: FileProcessingMetadata - Items returned (1)' is displayed, showing a single item with columns: file_name (String), columns, file_size_bytes, and file_type. The item's values are sample_data.csv, 5, 219, and .csv respectively. Below the screenshot, the text 'FileProcessingMetadata - 1 item returned: sample_data.csv, 5 columns, 219 bytes, .csv - confirming end-to-end pipeline execution' is displayed.

The CloudWatch log and DynamoDB entry matched exactly, confirming that the Lambda function processed the uploaded file correctly, extracted accurate metrics, and persisted the metadata without errors. The pipeline functioned as intended from first upload through to final storage.

Reflection and Insight

Before this assignment, I understood cloud computing as a concept but had not connected individual services into a working system. Building this pipeline changed that. The moment the CSV upload triggered Lambda automatically and the DynamoDB entry appeared on its own, event-driven architecture stopped being abstract and became something I could reason about from first principles.

The most significant thing I learned is the difference between knowing what a service does and understanding why it fits a specific problem. I initially assumed a continuously running server would be needed to monitor for uploads. Realising that Lambda eliminates that entirely running only when triggered made the value of serverless design genuinely clear, not just as a theoretical concept.

The IAM setup turned out to be more nuanced than expected. I had to think carefully about the distinction between a user policy and a service execution role. The Assignment user needed permissions to build the system; the Lambda role needed separate permissions to operate within it. These are two different identities with two different purposes and keeping them apart is not just good practice, it is how production cloud environments are actually secured.

One challenge I encountered was that the Assignment user lacked permission to create IAM roles, which caused Lambda setup to fail initially. Rather than simply broadening the user's permissions to work around it, I created the Lambda execution role manually as root and assigned it a more deliberate approach that reinforced the principle that least-privilege sometimes requires more upfront planning but produces a tighter and more defensible design.

If I were to extend this pipeline, I would add Amazon SNS notifications to alert a team whenever a file is processed, implement S3 versioning to maintain an upload history, and add error handling in Lambda for malformed CSV files. This assignment taught me that cloud architecture is not simply about connecting services it is about understanding why each connection exists, what it enables, and what risk it introduces or removes.